

Gemini Nouvelle discussion Rechercher dans les discussions Images Vidéos Bibliothèque README
Amélioration API FastAPI Spring DI Interface Implementation Jinja XSS: Causes and Prevention FastAPI et
Pydantic : Validation par Pattern Exposer un serveur SMTP avec Cloudflare Suffixage des DTOs : Entrée et
Sortie Résoudre l'erreur 422 FastAPI Chat Génération PDF Fiche Aéroport RAG ChromaDB ID Handling and
Batching Génération de Texte Aléatoire Absurde Ollama Scripting: Serve, Pull, Run Ollama Model Not Found
Error Socket.IO Code Bug Fixes Quadro K620 CUDA Support and Limitations Correction Erreurs Python
Dictionnaire Itération Correction des touches Curses pour la navigation Enter Key Codes: 13 vs 10 Correction
de l'initialisation des routes Récupérer le retour d'un événement Socket.IO Socket.IO `.off()` Event Listener
Removal Curses Socket.IO Chat Asyncio Fix Localtunnel et WebSockets avec Socket.io Probabilité : L'autre
enfant est une fille Latest JDK LTS Release Information Capturer Raikou, Entei, Suicune Pokémon Rouge Feu
Utiliser une Spritesheet pour un Jeu Découpage d'assets pixel art pour Phaser Phaser 3 Sprite Sheet Animation
Guide

Activité Seraing, Belgique D'après votre adresse IP Mettre à jour la position Conversation avec Gemini tu peux
me mettre mon readme au propre en corrigeant les incohérences

Exercice 1

1. Créer une api avec FastAPI

Dans l'api Ajouter

- model (sqlalchemy)

task

```
id int (auto inc)

name str

attribution_email str

status (in_progress, done)

end_date

start_date
```

- 1 controller task_controller

- add_task (post, /task)

entree: name, attribution_email, duration(nbjours)

à l'ajout de la tâche un email devra être envoyé à la personne qui doit effectuer la tâche

sauvegarde en db (sqlalchemy) par défaut la tâche devra avoir le status in_progress

- `get_tasks`: (get, /task)

entree: email, limit=10 (max 100), status, page=1 (int) (qui permettront de filtrer les taches)
- `change_task_status`: (put, /task/{id})

entree: status

modifier le status de la taches

on ne peut changer l'etat de la taches que si

si on est pas arrivé à la date d'echéance
- `delete_task`: (delete, /task/{id})

supprimer une tache

on ne peut supprimer une tache (done)

envoi d'un mail a la personne qui doit effectuer la tache pour signaler que la tache n'existe plus

Voici une version nettoyée, structurée et corrigée de votre README.

✂ Corrections apportées Types SQL / Pydantic explicites : Clarification des types pour `start_date` (DateTime / `now()`) et `end_date`.

Gestion de duration vs `end_date` : Explication explicite que `end_date` = `start_date` + duration lors de la création de la tâche.

Logique métier précisée :

Mise à jour du statut : Refus si `now()` > `end_date` (date d'échéance dépassée).

Suppression : Refus si la tâche est au statut done (suppression autorisée uniquement si `in_progress`).

Pagination : Correction du paramètre limit (par défaut 10, borné entre 1 et 100) et calcul de l'offset via page.

Pluriels REST : Harmonisation des endpoints sur `/tasks` plutôt que `/task` (bonne pratique REST).

Markdown



Exercice 1 - API de gestion de tâches avec FastAPI

Création d'une API RESTful de gestion de tâches avec **FastAPI**, **SQLAlchemy** et gestion d'envois d'e-mails.



Modèle de données (sqlalchemy)

Modèle Task

- **id** : Integer (Clé primaire, auto-incrémentée)

- **name** : `String` (Nom de la tâche)
 - **attribution_email** : `String` (Email de la personne assignée)
 - **status** : `Enum / String` (Valeurs possibles : `in_progress`, `done` — Par défaut : `in_progress`)
 - **start_date** : `DateTime` (Date de création / début — Générée automatiquement à `now()`)
 - **end_date** : `DateTime` (Date d'échéance — Calculée lors de la création : `start_date + duration`)
-

Contrôleur `TaskController` (`/tasks`)

1. Créer une tâche

- **Endpoint** : `POST /tasks`
 - **Paramètres d'entrée** (Body/Schema Pydantic) :
 - **name** : `str`
 - **attribution_email** : `EmailStr`
 - **duration** : `int` (Nombre de jours pour réaliser la tâche)
 - **Comportement & Logique métier** :
 - Définir **status** par défaut à `in_progress`.
 - Calculer **end_date** à partir de `start_date + duration`.
 - Sauvegarder la tâche en base de données.
 - **Email** : Envoyer un e-mail de notification à **attribution_email** pour l'informer de la nouvelle tâche.
-

2. Lister / Filtrer les tâches

- **Endpoint** : `GET /tasks`
 - **Paramètres de requête** (Query parameters) :
 - **email** : `str` (Optionnel - Filtrer par destinataire)
 - **status** : `str` (Optionnel - Filtrer par `in_progress` ou `done`)
 - **page** : `int` (Optionnel - Par défaut : `1`, min : `1`)
 - **limit** : `int` (Optionnel - Par défaut : `10`, max : `100`)
 - **Comportement** :
 - Retourne la liste des tâches paginée et filtrée selon les critères fournis.
-

3. Modifier le statut d'une tâche

- **Endpoint** : `PUT /tasks/{id}`
 - **Paramètres d'entrée** :
 - **id** : `int` (Path parameter)
 - **status** : `str` (Body parameter : `in_progress` ou `done`)
 - **Comportement & Logique métier** :
 - **Règle de gestion** : La modification du statut est autorisée **uniquement si la date actuelle n'a pas dépassé la date d'échéance** (`now() <= end_date`).
 - Si l'échéance est dépassée, renvoyer une erreur `400 Bad Request`.
-

4. Supprimer une tâche

- **Endpoint** : `DELETE /tasks/{id}`
- **Paramètres d'entrée** :
 - `id : int` (Path parameter)
- **Comportement & Logique métier** :
 - **Règle de gestion** : Une tâche ne **peut pas** être supprimée si son statut est `done`.
 - Si le statut est `in_progress` :
 - Supprimer la tâche de la base de données.
 - **Email** : Envoyer un e-mail à `attribution_email` pour notifier de l'annulation/suppression de la tâche. ok donne moi un md

Your Markdown file is ready

Icône MD README MD Ouvert Voici le fichier README.md généré et prêt à être utilisé dans votre projet.

tu as oublié les 404

Bien vu ! J'ai réintégré la gestion des erreurs 404 Not Found (lorsqu'une tâche demandée par son id n'existe pas en BDD) ainsi que la précision des 400 Bad Request et 422 Unprocessable Entity.

Voici le fichier README.md mis à jour :

Icône MD README MD Ouvert Markdown



Exercice 1 - API de gestion de tâches avec FastAPI

Création d'une API RESTful de gestion de tâches avec **FastAPI**, **SQLAlchemy** et gestion d'envois d'e-mails.



Modèle de données (sqlalchemy)

Modèle Task

- `id : Integer` (Clé primaire, auto-incrémentée)
- `name : String` (Nom de la tâche)
- `attribution_email : String` (Email de la personne assignée)
- `status : Enum / String` (Valeurs possibles : `in_progress`, `done` — Par défaut : `in_progress`)
- `start_date : DateTime` (Date de création / début — Générée automatiquement à `now()`)
- `end_date : DateTime` (Date d'échéance — Calculée lors de la création : `start_date + duration`)



Contrôleur TaskController (/tasks)

1. Créer une tâche

- **Endpoint** : `POST /tasks`
- **Paramètres d'entrée** (Body/Schema Pydantic) :
 - `name : str`
 - `attribution_email : EmailStr`
 - `duration : int` (Nombre de jours pour réaliser la tâche)

- **Comportement & Logique métier :**
 - Définir **status** par défaut à **in_progress**.
 - Calculer **end_date** à partir de **start_date + duration**.
 - Sauvegarder la tâche en base de données.
 - **Email** : Envoyer un e-mail de notification à **attribution_email** pour l'informer de la nouvelle tâche.
 - **Erreurs HTTP :**
 - **422 Unprocessable Entity** : Données d'entrée invalides (ex. email non valide, durée négative).
-

2. Lister / Filtrer les tâches

- **Endpoint** : **GET /tasks**
 - **Paramètres de requête** (Query parameters) :
 - **email** : **str** (Optionnel - Filtrer par destinataire)
 - **status** : **str** (Optionnel - Filtrer par **in_progress** ou **done**)
 - **page** : **int** (Optionnel - Par défaut : **1**, min : **1**)
 - **limit** : **int** (Optionnel - Par défaut : **10**, max : **100**)
 - **Comportement** :
 - Retourne la liste des tâches paginée et filtrée selon les critères fournis.
-

3. Modifier le statut d'une tâche

- **Endpoint** : **PUT /tasks/{id}**
 - **Paramètres d'entrée** :
 - **id** : **int** (Path parameter)
 - **status** : **str** (Body parameter : **in_progress** ou **done**)
 - **Comportement & Logique métier** :
 - **Règle de gestion** : La modification du statut est autorisée **uniquement si la date actuelle n'a pas dépassé la date d'échéance** (**now()** <= **end_date**).
 - **Erreurs HTTP** :
 - **404 Not Found** : Si la tâche correspondant à l'ID n'existe pas en base de données.
 - **400 Bad Request** : Si la date d'échéance est dépassée (**now()** > **end_date**).
-

4. Supprimer une tâche

- **Endpoint** : **DELETE /tasks/{id}**
- **Paramètres d'entrée** :
 - **id** : **int** (Path parameter)
- **Comportement & Logique métier** :
 - **Règle de gestion** : Une tâche ne **peut pas** être supprimée si son statut est **done**.
 - Si le statut est **in_progress** :
 - Supprimer la tâche de la base de données.
 - **Email** : Envoyer un e-mail à **attribution_email** pour notifier de l'annulation/suppression de la tâche.
- **Erreurs HTTP** :

- **404 Not Found** : Si la tâche correspondant à l'ID n'existe pas en base de données.
- **400 Bad Request** : Si la tâche est au statut **done**.

Vos discussions Brainstorm Consulting ne sont pas utilisées pour améliorer nos modèles. Gemini est une IA et peut se tromper, y compris sur des personnes. Votre confidentialité et Gemini s'ouvre dans une nouvelle fenêtre



Exercice 1 - API de gestion de tâches avec FastAPI

Création d'une API RESTful de gestion de tâches avec **FastAPI**, **SQLAlchemy** et gestion d'envois d'e-mails.



Modèle de données (sqlalchemy)

Modèle Task

- **id** : **Integer** (Clé primaire, auto-incrémentée)
 - **name** : **String** (Nom de la tâche)
 - **attribution_email** : **String** (Email de la personne assignée)
 - **status** : **Enum / String** (Valeurs possibles : **in_progress**, **done** — Par défaut : **in_progress**)
 - **start_date** : **DateTime** (Date de création / début — Générée automatiquement à **now()**)
 - **end_date** : **DateTime** (Date d'échéance — Calculée lors de la création : **start_date + duration**)
-



Contrôleur TaskController (/tasks)

1. Créer une tâche

- **Endpoint** : **POST /tasks**
 - **Paramètres d'entrée** (Body/Schema Pydantic) :
 - **name** : **str**
 - **attribution_email** : **EmailStr**
 - **duration** : **int** (Nombre de jours pour réaliser la tâche)
 - **Comportement & Logique métier** :
 - Définir **status** par défaut à **in_progress**.
 - Calculer **end_date** à partir de **start_date + duration**.
 - Sauvegarder la tâche en base de données.
 - **Email** : Envoyer un e-mail de notification à **attribution_email** pour l'informer de la nouvelle tâche.
 - **Erreurs HTTP** :
 - **422 Unprocessable Entity** : Données d'entrée invalides (ex. email non valide, durée négative).
-

2. Lister / Filtrer les tâches

- **Endpoint** : **GET /tasks**
- **Paramètres de requête** (Query parameters) :
 - **email** : **str** (Optionnel - Filtrer par destinataire)

- **status** : **str** (Optionnel - Filtrer par **in_progress** ou **done**)
 - **page** : **int** (Optionnel - Par défaut : **1**, min : **1**)
 - **limit** : **int** (Optionnel - Par défaut : **10**, max : **100**)
 - **Comportement** :
 - Retourne la liste des tâches paginée et filtrée selon les critères fournis.
-

3. Modifier le statut d'une tâche

- **Endpoint** : **PUT /tasks/{id}**
 - **Paramètres d'entrée** :
 - **id** : **int** (Path parameter)
 - **status** : **str** (Body parameter : **in_progress** ou **done**)
 - **Comportement & Logique métier** :
 - **Règle de gestion** : La modification du statut est autorisée **uniquement si la date actuelle n'a pas dépassé la date d'échéance** (**now()** <= **end_date**).
 - **Erreurs HTTP** :
 - **404 Not Found** : Si la tâche correspondant à l'ID n'existe pas en base de données.
 - **400 Bad Request** : Si la date d'échéance est dépassée (**now()** > **end_date**).
-

4. Supprimer une tâche

- **Endpoint** : **DELETE /tasks/{id}**
- **Paramètres d'entrée** :
 - **id** : **int** (Path parameter)
- **Comportement & Logique métier** :
 - **Règle de gestion** : Une tâche ne **peut pas** être supprimée si son statut est **done**.
 - Si le statut est **in_progress** :
 - Supprimer la tâche de la base de données.
 - **Email** : Envoyer un e-mail à **attribution_email** pour notifier de l'annulation/suppression de la tâche.
- **Erreurs HTTP** :
 - **404 Not Found** : Si la tâche correspondant à l'ID n'existe pas en base de données.
 - **400 Bad Request** : Si la tâche est au statut **done**.